NODEBASE · REFERENCE

OTP Email Verification Flow

End-to-end trace of the email-OTP path: from clicking “Create account”, through BullMQ queueing and SMTP delivery, to code verification and the redirect into the workflows dashboard. Includes the two failure modes hit during local setup and how they were diagnosed.

Next.js 15 (App Router) Better Auth 1.3 BullMQ 6 + Upstash Redis nodemailer tRPC v11

At a glance



The single most important structural fact: **the web request never sends the email.** It generates the code, stores it, and enqueues a job. Delivery happens in a separate OS process. A green `200` response therefore proves nothing about whether the email was sent.

PHASE A Sign up

1. `src/app/(auth)/signup/page.tsx`

   Server component. Calls `requireUnauth()`, then renders `<RegisterForm />`.

2. `src/lib/auth-utils.ts:15`

   `requireUnauth()` → `auth.api.getSession({ headers })`. If a session already exists it `redirect("/workflows")`, so signed-in users never see the signup form.

3. `src/features/auth/components/register-form.tsx`

   Client-side validation via `react-hook-form` + `zodResolver`. The schema at line 31 enforces: name present, valid email, password ≥ 8 chars, matching confirmation. No network request yet.

4. **`register-form.tsx:88` — `onSubmit`**

```
await authClient.signUp.email(
  { name, email, password, callbackURL: "/workflows" },
  {
    onSuccess: () =>
      router.push(`/verify-email?email=${encodeURIComponent(email)}`),
    onError: (ctx) => toast.error(ctx.error.message),
  },
);
```

   The client helper lives in `src/lib/auth-client.ts`. This POSTs to `/api/auth/sign-up/email`.

5 `src/app/api/auth/[...all]/route.ts:4`

The entire auth surface is a single catch-all handler:

```
import { toNextJsHandler } from "better-auth/next-js";
import { auth } from "@/lib/auth";

export const { POST, GET } = toNextJsHandler(auth);
```

Every endpoint in this document — `sign-up/email`, `email-otp/send-verification-otp`, `email-otp/verify-email` — funnels through this one file. If auth requests 404 or hang, look here first.

6 `src/lib/auth.ts:48` — the `betterAuth()` config

Three things happen in sequence:

a. Persistence. `prismaAdapter(prisma, { provider: "postgresql" })` (line 49) writes the `User` row with `emailVerified: false` (`prisma/schema.prisma:21`), plus an `Account` row for the password. Critically, **no `Session` row is created**, because `requireEmailVerification: true` (line 56).

b. Code generation. The `emailOTP` plugin (line 105) creates a 6-digit code (`otpLength: 6`, line 106) and stores it **hashed** (`storeOTP: "hashed"`, line 108) into the `Verification` model with `expiresAt = now + 300s` (`expiresIn: 300`, line 107). `overrideDefaultEmailVerification: true` (line 110) means this plugin takes over verification entirely, replacing Better Auth's built-in email flow.

c. Hand-off. `sendVerificationOTP` (line 111) → `enqueueOtpEmail({ email, otp, type })`.

7 `src/lib/queue/email-queue.ts:29` — `enqueueOtpEmail`

This is the hop that causes most confusion. It does not send anything:

• `getQueueConnection()` (`src/lib/queue/connection.ts:18`) returns an ioredis client built from `UPSTASH_REDIS_URL`, or `null` when unset.

• Queue constructed at line 15: `new Queue("email", { attempts: 3, backoff: { type: "exponential", delay: 2000 }, removeOnComplete: true, removeOnFail: 1000 })`.

• If Redis is *not* configured, line 33 falls back to sending inline. Because `UPSTASH_REDIS_URL` *is* set, it takes line 37 — `emailQueue.add("otp", data)` — and returns immediately.

8 Response returns `200`. `register-form.tsx:97-101` fires `onSuccess` and navigates to `/verify-email?email=...`. The browser is now sitting on the verify page while the email is still an unprocessed job in Redis.

PHASE B Delivery (separate OS process)

---

9 `src/queue/worker.ts`

A completely independent Node process, started by mprocs or by `npm run worker` (`package.json:13` = `dotenv -e .env -- tsx src/queue/worker.ts`). It is not started by the Next.js server and does not share its lifecycle.

```
const worker = new Worker<OtpEmailData>(
  "email",
  async (job) => { if (job.name === "otp") await sendOtpEmail(job.data); },
  { connection, concurrency: 2 },
);
```

Lines 7–13 hard-exit with a warning if `UPSTASH_REDIS_URL` is missing, so the worker cannot silently idle.

10 `src/lib/email.ts:78` — `sendOtpEmail`

The transport is created **once at module load** (lines 19–26) from `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS`, with `secure: port === 465`.

```
if (!transport) {
  if (process.env.NODE_ENV === "production") {
    throw new Error("SMTP is not configured (set SMTP_HOST, SMTP_USER, SMTP_PASS)");
  }
  console.log(`[email] dev fallback — OTP for ${email}: ${otp}`);
  return;
}
await transport.sendMail({ from, to: email, subject, text, html });
```

> **Operational consequence**
>
> Because the transport is module-level, it reads the environment exactly once. **Editing `.env` changes nothing until the worker process is restarted.** This is the single most common cause of "I fixed the password but it still fails."

11 nodemailer hands off to `smtp.gmail.com:587`, which either accepts and delivers, or rejects. On failure, `worker.ts:34` (`worker.on("failed")`) logs the reason to the worker console, the job retries 3× with exponential backoff, then lands in the `failed` sorted set in Redis.

PHASE C Entering the code

12 `src/app/(auth)/verify-email/page.tsx`

Awaits `searchParams` and passes the email down to `<VerifyEmailForm email={...} />` as a prop.

13 `src/features/auth/components/verify-email-form.tsx`

`InputOTP maxLength={6}` with `onComplete={verify}` (line 161) auto-submits the instant the sixth digit lands — no button press required. `verify()` (line 84) POSTs to `/api/auth/email-otp/verify-email`, back through the same `[...all]/route.ts` into the `emailOTP` plugin.

The plugin hashes the submitted code and compares it against `Verification.value`. `allowedAttempts: 5` (line 109) caps incorrect guesses.

On a match it does three things:

• sets `User.emailVerified = true`
• deletes the `Verification` row
• because `autoSignInAfterVerification: true` (line 62) — **creates the `Session` row and sets the session cookie**

Lines 117–118 then run `toast.success("Email verified!")` followed by `router.push("/workflows")`.

PHASE D Landing on workflows

14 `src/app/workflows/page.tsx`

`requireAuth()` → `auth.api.getSession({ headers })` reads the cookie set in step 13, finds the session, and permits entry. From there the standard pattern applies: `prefetch` → `<HydrateClient />` → `useSuspenseQuery`. Without that session the request is redirected to `/login` (observed as `307`).

The resend path

`verify-email-form.tsx:52` `sendOtp()` → `POST /api/auth/email-otp/send-verification-otp` → rejoins the chain from step 6c onward. Two independent guards sit in front of it:

GUARD	MECHANISM
`otpCooldownPlugin` `auth.ts:19-46`	A `before` hook that runs *before* Better Auth generates a code, so a blocked request neither burns the queue nor invalidates the code the user already holds. This ordering is the entire point of the plugin.
`otpSendAllowed` `otp-limit.ts:46`	Upstash `Ratelimit.slidingWindow(RATE_LIMIT_OTP_PER_HOUR ?? 5, "1 h")` keyed on the lowercased email, plus a 60-second idempotency cooldown via `acquireIdempotencyKey`. Returns `429` when blocked.
`auth.ts:81`	Global `rateLimit.customRules`: 3 requests / 60s on the send endpoint, 8 / 300s on `/email-otp/verify-email`. Baseline across all endpoints is 20 / 60s.

Failure modes encountered, and how they were found

1. `403 Invalid origin: http://localhost:3000`

Root cause

`src/lib/auth.ts:74` read `trustedOrigins: [process.env.NEXT_PUBLIC_APP_URL!]`. In `.env` that variable held the ngrok tunnel URL, so browsing on `localhost:3000` was untrusted and Better Auth rejected the request before any code was generated.

Fix. Add the local origins and normalise trailing slashes, since `getTrustedOrigins` compares against `new URL(baseURL).origin` while the env values carried a trailing slash:

```
trustedOrigins: [
  process.env.NEXT_PUBLIC_APP_URL,
  process.env.BETTER_AUTH_URL,
  "http://localhost:3000",
  "http://127.0.0.1:3000",
]
  .filter((origin): origin is string => Boolean(origin))
  .map((origin) => origin.replace(/\/+$/, "")),
```

Verification. Requests were replayed with real cookies against three origins:

ORIGIN HEADER	STATUS	MEANING
`http://localhost:3000`	`401`	Passed the origin check; rejected on credentials as expected
`https://thoughtful-...ngrok-free.dev`	`401`	Still trusted — tunnel flow unbroken
`https://evil.example.com`	`403 INVALID_ORIGIN`	CSRF protection intact — the fix did not weaken it

Two testing traps worth knowing

1. Better Auth's `originCheckMiddleware` only validates when `headers.has("cookie")` is true. Requests from curl or Postman *without* cookies skip the check entirely and return a misleading `401` instead of `403`.
2. `Invoke-WebRequest -Headers @{Cookie=...}` silently drops the header in PowerShell 5.1. Use `curl.exe -H "Cookie: ..."` for origin testing.
3. The 20-requests-per-60s rate limit is keyed by IP and shared with the browser, so scripted testing exhausts it and yields `429` that masks the real result. Wait out the window before drawing conclusions.

2. No email arrives — `535 BadCredentials`

The origin fix let sign-up return `200` and the code was generated, but no email arrived. Three candidates existed: origin, worker not running, bad SMTP credentials.

Reading the queue directly was what resolved it. Rather than adding logging, the BullMQ internal keys in Redis were inspected:

```
bull:email:wait   = 0     ← consumed by the worker
bull:email:active = 0
bull:email:failed = 10    ← and it died
completed         = 0
```

Then the job hash was dumped with `hgetall` to retrieve `failedReason`, which contained the server's own diagnosis:

```
job 11 | {"email":"hokip78055@ncleap.com","otp":"244455","type":"email-verification"}
failedReason: Invalid login: 535-5.7.8 Username and Password not accepted
```

The worker *was* running (confirmed via `Win32_Process`, showing mprocs had `worker` and `cron-worker` alive), so that theory was wrong and the credential was the sole cause. A stale-worker hypothesis was then eliminated by comparing `.env` mtime against the worker process start time.

Fix

Gmail rejects account passwords over SMTP and requires a 16-character **App Password** (Google account → Security → 2-Step Verification → App passwords). The configured value was 19 characters — an app password is always exactly 16, which makes the length itself a reliable diagnostic. Set `SMTP_PASS` to the 16-character value without spaces, then **restart the worker process**.

Next failure in the chain

`EMAIL_FROM` was `"Nodebase <no-reply@nodebase.app>"` while `SMTP_USER` was `ishanksharma4444@gmail.com`. Gmail rewrites or rejects a `From` header that does not match the authenticated account, so align them until a real sending domain exists.

3. mprocs: `os error 193` on Windows

All five processes failed instantly with `%1 is not a valid Win32 application`. mprocs resolved `npm` to `C:\Program Files\nodejs\npm` — a POSIX shell script Windows cannot execute — rather than to `npm.cmd`. Changing every `cmd` array in `mprocs.yaml` to `npm.cmd` resolves it. Note the trade-off: that file is now Windows-only, and `cmd` must remain an array (the string form is rejected by the schema).

Key configuration reference

VARIABLE	USED BY	NOTES
NEXT_PUBLIC_APP_URL	auth.ts:74	Primary trusted origin. Public tunnel URL in this environment.
BETTER_AUTH_URL	Better Auth	Base URL; also folded into `trustedOrigins`. Must not carry a trailing slash.
UPSTASH_REDIS_URL	connection.ts:4	ioredis-compatible URL for BullMQ. When unset, `enqueueOtpEmail` silently degrades to inline sending.
UPSTASH_REDIS_REST_URL / _TOKEN	otp-limit.ts:12	REST endpoint used by the Upstash Ratelimit sliding window.
SMTP_HOST / _PORT	email.ts:9	`secure` is inferred: `port === 465`. Use 587 for STARTTLS.
SMTP_USER / SMTP_PASS	email.ts:11	Gmail requires a 16-character App Password, not the account password.
EMAIL_FROM	email.ts:13	Must match the authenticated SMTP account or Gmail rewrites it.
RATE_LIMIT_OTP_PER_HOUR	otp-limit.ts:33	Default 5 per rolling hour, per email address.
POLAR_ACCESS_TOKEN	polar.ts	Currently invalid (`invalid_token`). Billing and `premiumProcedure` fail independently of the email flow.

Operational note on the queue

Upstash Redis reports `Eviction policy is optimistic-volatile. It should be "noeviction"`. This was not the cause of any failure above — the jobs persisted in the `failed` set — but under memory pressure Redis may silently discard BullMQ job keys. Consider moving the email queue off Upstash before relying on it in production.

Data at rest — what is stored, and where

Between clicking “Create account” and submitting the code, **three tables are written and a fourth deliberately is not**. Every figure below was read live from the project’s Neon database for a real account, not inferred from schema definitions.

LIVE SNAPSHOT — ACCOUNT MID-VERIFICATION

```
=== USER TABLE ===
id            : Nxq4q261jT2IjFUvYkUkL7epeWennuOE
email         : hokip78055@ncleap.com
emailVerified : false   // ← still unverified
createdAt     : 2026-09-27T08:58:42.942Z

=== ACCOUNT TABLE ===
providerId    : credential    // 'credential' = email+password
accountId     : Nxq4q261jT2IjFUvYkUkL7epeWennuOE
password      : 19d23ab73d8883724cc9… [len=161]
createdAt     : 2026-09-27T08:58:47.093Z
```

```
=== SESSION TABLE ===
count         : 0            // ← nothing can sign in yet
```

Where the email and password are stored

- **Email — plaintext, in the `user` table.**

  `prisma/schema.prisma:20` declares `email String`, required and indexed via `@@unique([email])` (line 30). It has to be stored in the clear: it is the lookup key for sign-in, and the OTP identifier is derived from it. Anyone with database access can read every address — treat the table accordingly.

- **Password — scrypt-hashed, in the `account` table, never on `user`.**

  This is the detail most implementations get wrong. `Account.password` (`schema.prisma:61`) is a *nullable* column on a **separate table**, and the credential is identified by a *row* — `providerId = "credential"` — rather than by a column on the user. **The `user` table has no password column at all.**

  Verified against the installed package, `crypto-CFUhAR9W.mjs:132-136`:

  ```
  const hashPassword = async (password) => {
    const salt = hex.encode(crypto.getRandomValues(new Uint8Array(16)));
    const key = await generateKey(password, salt);          // scrypt
    return `${salt}:${hex.encode(key)}`;
  };

  const verifyPassword = async ({ hash, password }) => {
    const [salt, key] = hash.split(":");
    return constantTimeEqual(await generateKey(password, salt), hexToBytes(key));
  };
  ```

  So the stored value is `<32-hex-char salt>:<128 hex chars>` — a 16-byte random salt plus a 64-byte derived key, giving the 161 characters measured above (32 + 1 + 128). Parameters are fixed in the library at:

PARAMETER	VALUE	PURPOSE
N	16384	CPU/memory cost — the primary defense against brute force.
r	16	Block size.
p	1	Parallelization.
dkLen	64	Derived key length in bytes → 128 hex characters.

  The password is `normalize("NFKC")`-ed before derivation, so visually identical passwords written with different Unicode forms still match. Comparison uses `constantTimeEqual`, which is timing-safe — it does not leak the hash through response latency.

  The same `account` table is why one human can hold several credentials. In this database the same user has two `providerId = "google"` rows alongside the credential row, each with `password = null` and `accountId ≠ userId` (Google's subject ID instead of the internal one). One `user`, three `account` rows, no password on any OAuth row.

Why a separate `account` table, rather than a password column on `user`

A reasonable first instinct is to add `password` to `user` and stop. That works right up until a user signs in with Google, and it costs a structural rewrite at exactly the moment the schema is hardest to change. The separate

table is not a quirk of this library — Better Auth, NextAuth/Auth.js, Lucia, and Django all split identity from credential. Six concrete reasons, measured against this project's own data:

REASON	WHAT BREAKS WITHOUT THE SPLIT
One user, N credentials	A single column holds exactly one value. The account above needs a password *and* two Google logins — `providerId` is `credential`, `google`, `google`. With `user.password` the second provider has nowhere to go, so the second sign-in is impossible. This is the primary reason, and it is not hypothetical: it is the two `google` rows already in the database.
Provider-specific columns stay off `user`	`account` also carries six nullable OAuth columns — `accessToken`, `refreshToken`, `idToken`, `accessTokenExpiresAt`, `refreshTokenExpiresAt`, `scope` (`schema.prisma:55-60`) — all meaningless for a password login. Moving them onto `user` means six permanently-NULL columns on every password-only account, and it mixes token lifecycle into the identity row.
The right key gets the uniqueness constraint	The natural identity of an account link is `(providerId, accountId)`, which is what the table is indexed on. `accountId` is Better Auth's own value for that provider: the internal user id for `credential`, Google's `sub` for Google (confirmed: `accountId ≠ userId` on both `google` rows). Keeping `accountId` distinct from `userId` is what lets a third-party identifier be stored verbatim without colliding with your own id space.
Credentials change independently of identity	Unlinking a Google account, rotating a password, or revoking one provider's tokens is a single-row `DELETE` or `UPDATE` that never touches the user record. On a combined table, disconnecting a provider risks clobbering the user row or invalidating unrelated sessions.
Account linking becomes an insert	When a user signs in with a provider they have used before, you look up `(providerId, accountId)` and reuse the existing `userId`. With a flat `user` table you have no way to record that two credentials belong to the same human — you would either merge rows destructively or duplicate the user.
Blast radius, enforced by the schema	Because `user` has no `password` column at all, no ad-hoc `SELECT * FROM "user"` — a support query, an analytics export, a BI tool, a log line — can leak a credential hash. A structural guarantee beats remembering to avoid selecting a column.

The honest cost

Resolving "which credential belongs to this session" needs a second read, so authenticated requests pay one extra indexed query, and the password path is `user → account` rather than a single row. At this scale that is the right trade. The split is also why `account.password` is *nullable* and why "this user has no password" is a legitimate, expected state rather than a data-integrity error.

What the `verification` table is for

Despite the singular name, this is Better Auth's **generic store for any short-lived, single-use secret** — not an email-verification-specific table. It has only four meaningful columns: `identifier` (what the secret belongs to), `value` (the secret, hashed), `expiresAt`, and `createdAt`. The physical table is lowercase `verification` via `@@map("verification")`.

ACTUAL ROWS FOR THE ACCOUNT ABOVE

```
identifier : email-verification-otp-hokip78055@ncleap.com
value      : hd5Ha76yKxa00lyVBf… [len=45]   // hashed OTP, not 244455
expiresAt  : 2026-09-27T09:03:47.461Z   (EXPIRED)
createdAt  : 2026-09-27T08:58:47.461Z
```

```
// 4 rows total in the table; all expired
```

COLUMN	WHAT THE MEASUREMENT SHOWS
`identifier`	A **namespaced key**, not a bare email: `email-verification-otp-<email>`. A sign-in OTP would use a different prefix in the same table — which is how one four-column table serves every auth feature. Practical consequence: querying by the bare address returns nothing.
`value`	45 characters, and **not the code that was emailed**. The delivered OTP was `244455` (recorded in BullMQ job 11); only `storeOTP: "hashed"` output (`auth.ts:108`) is persisted. A leaked database dump therefore cannot be used to complete a verification.
`expiresAt`	Exactly `createdAt + 300s`, matching `expiresIn: 300` (`auth.ts:107`). Expiry is enforced by timestamp, not by a cleanup job.

The table is never pruned

This account has **three** rows sharing the identifier `email-verification-otp-hokip78055@ncleap.com`, from three separate attempts. Resending does not delete the prior row — it inserts another. They are ignored only because `expiresAt` has passed, which means correctness does not depend on cleanup while **table size does**. A periodic delete of expired rows, or a TTL on the index, is worth adding before this runs at real volume.

Every purpose the `verification` table serves

The table is the single scratchpad for **every short-lived secret in the auth system**: OTPs, reset links, OAuth authorization codes, 2FA handshakes. All of them share one shape — a *who*, a *what*, and an *until* — so they share four columns. Each feature namespaces its `identifier` so they can never collide, which is why the column is a composite key and not the user id.

Below, extracted from the installed `better-auth` package rather than from its documentation. ACTIVE rows are reachable with this project's current configuration; AVAILABLE rows require a plugin or option that is not enabled here.

FEATURE	IDENTIFIER TEMPLATE	SEP.	VALUE STORED	STATUS HERE
Email verification	`email-verification-otp-<email>`	-	hashed 6-digit OTP	**ACTIVE** Live in the database today
Sign-in OTP	`sign-in-otp-<email>`	-	hashed 6-digit OTP	**ACTIVE** Reachable via `sendOnSignIn: true` (auth.ts:60)
Password reset link	`reset-password:<token>`	:	random token	**AVAILABLE** `emailAndPassword.forgetPassword` unset
Password reset OTP	`forget-password-otp-<email>`	-	hashed OTP	**AVAILABLE** needs the emailOTP `forgetPassword` option
Account deletion	`delete-account-<token>`	-	random token	**AVAILABLE**
Two-factor challenge	`2fa-<20-char random>`	-	**user id**	**AVAILABLE**
2FA code check	`2fa-otp-<key>`	-	hashed 2FA code	**AVAILABLE**
Magic link	`<raw token, unprefixed>`	–	token	**AVAILABLE**
Single-use API token	`one-time-token:<token>`	:	token	**AVAILABLE**
SIWE wallet auth	`siwe:<address>:<chainId>`	:	message & signature	**AVAILABLE**
Phone OTP / reset	`<phoneNumber>-request-password-reset`	-	hashed code	**AVAILABLE**
OAuth/OIDC auth code	`<32-char code, unprefixed>`	–	**JSON blob**	**AVAILABLE** via the OIDC provider plugin

Three patterns worth noticing

1. Three different separator conventions. Hyphen for OTP features (`email-verification-otp-`), colon for token-shaped payloads (`reset-password:` , `siwe:`), and nothing at all where the token is already high-entropy. This is why the `identifier` column can be indexed without a length bound and why a query for `identifier = 'hokip78055@ncleap.com'` returns nothing — the prefix is mandatory.

2. The `value` column is not always a hash. It is hashed for anything a human types, but it holds the *user id* for the 2FA handshake and a *JSON payload* for OAuth codes (`clientId` , `redirectURI` , `scope` , `userId` , `authTime` , `codeChallenge` , `nonce`). Hashed is a property of the *feature*, not a guarantee of the column — so the column is `text` with a length of at least 32, deliberately wide.

3. One feature can use two identifiers. The 2FA flow writes a `2fa-<random>` row carrying the user id, then a separate `2fa-otp-<key>` row carrying the hashed code. They are two different rows in two different phases, which is the clearest illustration that this table is a general-purpose key/value store with a TTL, not an email feature.

And one that is deliberately bypassed here

Better Auth ships a default verification flow that stores `email-verification:<token>` — colon, no `otp` segment. **This project never uses it.** `overrideDefaultEmailVerification: true` (`auth.ts:117`) replaces that endpoint with the OTP flow above, which is why the live row reads `email-verification-otp-` and not `email-verification:`. If you ever remove that flag, the identifier format changes and any queries you wrote against it will silently match nothing.

No session exists until verification

The single most important consequence of `requireEmailVerification: true` (`auth.ts:56`): at signup **no `session` row is created and no cookie is issued** — measured as `count = 0` above.

The account is real, committed, and password-verifiable, but there is no way to act on it. Because `DATABASE_URL` points at a live Neon instance rather than an ephemeral store, the `user` and `account` rows persist as genuine data for an account that nobody can yet sign into. Verification flips `emailVerified` to `true` and only then does `autoSignInAfterVerification: true` (`auth.ts:59`) create the session.

STEP	USER	ACCOUNT	VERIFICATION	SESSION
Signup submitted	row created `emailVerified: false`	row created `password` = scrypt hash	—	—
OTP generated	—	—	row created hashed, +300s	—
Email delivered	—	—	expires naturally	—
Code submitted	`emailVerified: true`	—	row deleted	**row created + cookie set**
Session used	—	—	—	validated on every request

Current database totals, for scale: 18 users (11 verified, 7 unverified), 30 sessions, 4 verification rows — all expired.

Method note

The snapshot, the `verification` rows, the hash structure, and the session count were all read directly from the project's Neon database using Prisma. The scrypt parameters and the `constantTimeEqual` comparison were read from the installed `better-auth` package rather than from documentation. Nothing on this page is assumed.

— end of document —